A Foundry agent: grounding, MCP servers, your own function, and a second agent

Handbook for trainers and students of the AI Engineer (Data) course. About 30 minutes to set up, 40 minutes to show. Tested on 14 and 15 October 2026. Parts 1, 2 and 5 are done in the Microsoft Foundry portal, ai.azure.com. Parts 3 and 4 also use the Azure portal, portal.azure.com, and VS Code.

Before you start

Sign in to ai.azure.com with the account that owns the Foundry project. If you see *Selected user account does not exist in tenant*, the project belongs to a different directory: pick the other account, or switch directory at the top right. The portal's labels change often, so click through these steps once before the session.

Use gpt-4.1-mini, not gpt-5-mini. In testing, gpt-5-mini repeated its input instead of writing a reply, and hit rate limits. **Do not add the Web search tool:** it lets the agent answer from the internet, which defeats the grounding the demo is about.

Part 1. Grounding on the refund policy

Setup

1. Deploy a model in your project: **gpt-4.1-mini**.
2. Create an agent and give it these instructions:

   > *Answer only using the attached document. If the document does not contain the answer, reply exactly: I could not find this in the refund policy. Quote the section you used.*

3. Add the **File search** tool and upload **refund_policy_v3_2024.pdf**. Foundry splits the file into pieces and indexes them for search, which is what the class built by hand in Labs 1 to 3. The file is in the course repository at **data/raw/**.

Questions to ask in the playground, in this order

Question	What it shows
How many refund claims can a customer make in thirty days?	Grounded answer: three in any rolling thirty-day period, with a citation to the document.
How long after delivery can a customer make a claim?	Grounded answer: 48 hours after the delivery time, with a citation.
What is the refund policy for grocery orders in Singapore?	It refuses, or quotes the policy's Scope section: it applies to orders in India. Either is grounded. This is the Lab 4 Singapore question.
What was the refund rate in Q1 2024?	It refuses, because that number is in a different FreshCart document that was not uploaded.

Finish with a contrast. Remove the file and the “only” instruction, then ask the Singapore question again. The model will likely give a confident general answer about refund policies that FreshCart never wrote. The same question with and without grounding makes the point quickly.

Link it back to the labs. The citation students see in the playground is the same idea as the source identifiers in Lab 4, but Foundry adds it for you. It does not check the citation in code the way Lab 4 did, so tell the class that.

Part 2. Add an MCP server

Use Microsoft's public **Microsoft Learn MCP server**. Microsoft runs it, it searches Microsoft's own documentation, and it needs no sign-in, so there is nothing to build or host. MCP (Model Context Protocol) is a standard way for a program to offer tools to any AI app.

Setup

1. Open the agent, go to **Tools**, add a tool and choose **MCP**.
2. Enter a name such as **microsoft_learn** and the server address **https://learn.microsoft.com/api/mcp**.
3. In the tool's **Configure** menu, leave approval on **Never auto-approve tools**. Each time the agent wants to use the server, the playground shows the exact search it wants to run, with **Approve** and **Deny**. Open the small arrow on Approve and choose **Approve once**.
4. Update the agent's instructions so it knows when to use which source:

> *You answer two kinds of questions. 1) FreshCart refund policy questions: answer only from the attached refund policy document, and quote the section you used. If the document does not answer the question, reply exactly: I could not find this in the refund policy. Do not explain what the document does cover, and never guess. 2) Questions about Microsoft or Azure products: use the Microsoft Learn MCP server tool and include links to the pages you used. For anything else, reply exactly: I can only answer questions about FreshCart's refund policy or Microsoft products.*

Demo questions

Question	What it shows
How many refund claims can a customer make in thirty days?	Answered from the policy document, as in Part 1.
What does the semantic ranker in Azure AI Search do?	The agent asks to run microsoft_docs_search. After you approve, it explains semantic ranking (reranks the top results, adds captions and answers) with a learn.microsoft.com link.
What is the capital of France?	Refused: neither source covers it, and the instructions say not to answer from memory.

What the class takes away. Two kinds of grounding side by side. The policy document is your own data, which Foundry indexed for you. The MCP server is someone else's data, reached live through the standard protocol.

Part 3. Your own data: a Python function as an OpenAPI tool

Parts 1 and 2 used a document and Microsoft's data. Now the agent gets FreshCart's own numbers. You write one small Python function, put it in Azure, and give it to the agent as a tool. The function, **get_metric**, returns one official number for one month of 2024: revenue, order count or refund claims. It reads them from **freshcart.db**, the same warehouse file as Lab 5, which is uploaded together with the code.

Why a function and not the raw table. The business rules are written once, in code: test accounts are always left out, and revenue counts delivered orders only. The agent cannot forget a rule or write a wrong query, because it only chooses which number and which month.

What is in the code folder

The folder is **lab5_foundry** in the course repository. Besides this handbook and a short README, it has five files:

File	What it does
function_app.py	The code. One shared lookup, offered two ways: a web address (this part) and an MCP tool (Part 4).
freshcart.db	The warehouse data. The code opens it read-only, so the function can never change it.
requirements.txt	The one package Azure installs: azure-functions.
host.json	Azure settings. It loads Microsoft's preview add-on that Part 4 needs for MCP.
openapi.json	The description of the web address that you paste into Foundry. It tells the agent what the tool does and what to send it.

The part of the code students should see. The function always answers with status 200 and puts any problem in the reply, because Foundry stops the whole agent run when a tool returns an error code:

```
@app.route(route="get_metric", methods=["GET"])
def get_metric(req):
    body = lookup(req.params.get("metric"), req.params.get("month"))
    return func.HttpResponse(json.dumps(body), status_code=200,
                             mimetype="application/json")
```

Step A. Create the Function App (portal, 5 minutes)

1. Go to **portal.azure.com**, click **Create a resource**, then **Function App**.
2. Hosting option: **Flex Consumption**. You pay per call, which is close to nothing when it sits idle.
3. Resource group: pick or create one, for example **refund-demo-rg**. Name: any free name, for example **freshcart-tools-1015**. Region: any region with free capacity; ours is **West US 3**.
4. Runtime stack **Python**, version **3.11**. Leave everything else as it is. Click **Review + create**, then **Create**, and wait about 2 minutes.
5. Open the new app. On **Overview**, copy the **Default domain**. It usually has a random part after the name, like **freshcart-tools-1015-<random part>.westus3-01.azurewebsites.net**. Do not type the address from the app's name; copy it.

Step B. Deploy the code from VS Code (2 minutes)

1. One-time setup: in VS Code, install the extension **Azure Functions** (by Microsoft). Click the Azure icon in the left bar and choose **Sign in to Azure**.
2. **File**, **Open Folder**, and open the **lab5_foundry** folder.
3. In the Azure panel, open **Resources**, your subscription, **Function App**. Right-click your app and choose **Deploy to Function App**. Click **Deploy** to confirm.
4. Wait for the message that the deployment completed, about 2 minutes, because Azure installs the package.
5. In the portal, open the app's **Functions** list. You should see two functions: **get_metric** and **get_metric_mcp**. The second one is for Part 4.

   The same deploy as a command, run from inside lab5_foundry, for anyone without VS Code:

```
zip -q package.zip function_app.py host.json requirements.txt freshcart.db
az functionapp deployment source config-zip -g refund-demo-rg \
  -n freshcart-tools-1015 --src package.zip --build-remote true
```

Step C. Test the function and get its key

1. Open **get_metric** in the Functions list and click **Test/Run**. Method **GET**, add two query values: **metric** = **revenue** and **month** = **2024-03**. Click **Run**. The reply should contain **"value": 531043.03**.
2. Run it again with **metric** = **average_delivery_time**. The reply is a message, *metric must be revenue, order_count or refund_claims*, not an error.
3. Get the key: in the app's left menu, go to **Functions**, then **App keys**. Under **Host keys**, copy the value of **default**. Keep it off the projector: anyone with the key can call your function.

Expected answers, checked against the live app on 15 October 2026:

metric and month	Value
revenue, 2024-03	531,043.03
revenue, 2024-02	522,100.56
order_count, 2024-05	450
refund_claims, 2024-03	21
average_delivery_time, 2024-06	A message: the function has no such measure.
any measure, with no key	Refused by Azure with 401 (not signed in).

Step D. Add the OpenAPI tool in Foundry

1. Open **openapi.json**. In **servers**, **url**, replace **YOUR-APP-ADDRESS** with your app's address from Step A, so it reads like **https://freshcart-tools-1015-<random part>.westus3-01.azurewebsites.net/api**.
2. Check that the **operationId** is **get_metric_openapi**. This is the function name the agent sees. It must differ from the MCP tool's name in Part 4, or the agent cannot tell the two tools apart.
3. In Foundry, open your agent (ours is **FirstAgent**), then **Tools**, **Add**, **OpenAPI tool**.
4. Name: **freshcart_metrics**. Paste the whole contents of openapi.json.
5. Authentication: **API key**, as a connection. Header name **x-functions-key**, value: the **default** key from Step C.
6. Add a third line to the agent's instructions:

   3) Questions about FreshCart's revenue, order counts or refund claims for a month of 2024: use the get_metric_openapi tool and give the number with its month. Never estimate a number.

7. Click **Save** at the top right. The playground uses the saved version only.

Demo questions

Question	What it shows
What was revenue in March 2024?	One tool call, 531,043.03. No approval step: OpenAPI tools run straight away.
How many orders were placed in May 2024?	One call, 450.
Compare revenue in February and March 2024.	Two calls, 522,100.56 and 531,043.03, then the comparison.
What was the average delivery time in June 2024?	The function replies that it has no such measure, and the agent says so instead of guessing.

Show the class the call. Click **Traces** under the answer: it shows the exact metric and month the agent sent, and the reply from your function.

Part 4. The same function as your own MCP server

In Part 2 the agent used Microsoft's MCP server. Now your Function App becomes an MCP server too. The code already contains a second way in to the same lookup, so there is nothing new to write. Azure offers every MCP function in the app at one address, **/runtime/webhooks/mcp**.

The MCP part of the code. Azure reads the tool's name, description and inputs from it, and sends the agent's inputs in **context**:

```
@app.generic_trigger(arg_name="context", type="mcpToolTrigger",
    toolName="get_metric", description="Return one official FreshCart number ...",
    toolProperties=json.dumps([... "metric" ..., ... "month" ...]))
def get_metric_mcp(context) -> str:
    arguments = json.loads(context).get("arguments", {})
    return json.dumps(lookup(arguments.get("metric"), arguments.get("month")))
```

The setting that makes it load is in **host.json**. MCP for Azure Functions is still a preview, so the app must use Microsoft's preview package:

```
"extensionBundle": {
  "id": "Microsoft.Azure.Functions.ExtensionBundle.Experimental",
  "version": "[4.*, 5.0.0)"
}
```

Step A. Check the MCP function is live

1. If you deployed in Part 3, it is already there: the Functions list shows **get_metric_mcp**. If it shows only get_metric, host.json was not deployed with the preview package; fix it and deploy again.
2. Get the MCP key. This is a **different** key from Part 3. In the app's left menu, go to **Functions**, then **App keys**. Under **System keys**, copy the value of **mcp_extension**.

The key mistake to avoid. The default key from Part 3 does not open the MCP address. We tested it: the default key gets 403 (not allowed) and a wrong or missing key gets 401 (not signed in). Foundry shows this as *Authentication failed when connecting to the MCP server ... 401 (Unauthorized)*.

Step B. Add the MCP tool in Foundry

1. In your agent, open **Tools**, **Add**, **MCP**.
2. Name: **freshcart-mcp**.
3. Server address: put it together from three pieces, in this order, all from your Function App's page in the Azure portal. The mcp_extension key only exists after you deploy the code, so deploy first.

Piece	Where to get it
1. The app's address, with **https://** in front	The app's **Overview** page: copy the **Default domain**, for example **freshcart-tools-1015-<random part>.westus3-01.azurewebsites.net**.
2. **/runtime/webhooks/mcp?code=**	Type it as shown. It is the same for every Function App.
3. The key	The app's left menu: **Functions**, then **App keys**. Under **System keys**, copy the value of **mcp_extension**.

The finished address reads like **https://freshcart-tools-1015-<random part>.westus3-01.azurewebsites.net/runtime/webhooks/mcp?code=<mcp_extension key>**, with no spaces anywhere.

4. Authentication: **none** (unauthenticated), because the key is already in the address. Do not show this address on the projector.
5. Approval: leave **Never auto-approve tools**, as in Part 2.
6. Click **Save**. Until you do, the Save button stays highlighted and the tool is not in use.

The other way to pass the key. Keep the address without **?code=** and choose key authentication with a custom header instead. The header name must be exactly **x-functions-key**, and the value is the mcp_extension key with no spaces before or after it. Both ways were tested and both return the right answer.

Step C. Test it

1. Ask **What was revenue in March 2024?**

2. The playground shows the call it wants to make, **get_metric** with **revenue** and **2024-03**, and the buttons **Approve** and **Deny**. Under it you see the labels **mcp_list_tools** (the agent asked the server which tools it has) and **mcp_approval_request**.
3. Click **Approve**. The answer is **$531,043.03, excluding test accounts**, and the label under it names the tool it came from: the name you gave the MCP tool, such as **freshcart-mcp**.

How the agent chooses between the two tools

Foundry has no fixed rule. The model reads each tool's function name and description, plus the agent's instructions, and picks the one that fits the question best. It does **not** see the labels you see in the portal, such as freshcart_metrics.

We saw this in testing. Both tools were first named **get_metric**. Asking "use the freshcart_metrics tool" still sent the question to the MCP tool, because the model saw two functions with the same name and could not tell them apart. The fix is to give the OpenAPI function its own name, **get_metric_openapi**, as in Part 3 Step D. To steer it further, add a line to the instructions, for example: *use get_metric_openapi for FreshCart numbers*.

You see	Which tool answered
Approve and Deny buttons, the label mcp_approval_request	The MCP tool. Only MCP tools ask for approval here.
No approval step	The OpenAPI tool.
Either one	Click Traces under the answer to see the exact call and reply.

What the class takes away. One function, two ways in. OpenAPI describes a web address for one app. MCP is a standard any AI app can connect to and ask "which tools do you have?". The business rules live in one place, the shared lookup, so both ways give the same number.

Part 5. A second agent: agent to agent

A second agent, **Replywriter**, turns the policy agent's answer into an email to the customer. A **workflow** runs the two in order and passes the first agent's answer to the second. The class sees one agent hand its work to another.

Setup

1. Create a new agent, **Replywriter**, on **gpt-4.1-mini**, with **no tools** and these instructions:

   > *The message you receive is NOT from a customer. It is an internal answer written by FreshCart's policy agent. Rewrite it as an email to the customer: start with "Dear customer,", use at most 5 sentences, keep any policy rule or section it quotes, and end with "FreshCart Support". Use only the facts in the answer you are given and never add a policy detail of your own. If the answer says the information could not be found, apologise and say a support agent will follow up.*

2. Go to **Build**, **Agents**, **Workflows**, and create a **sequential** workflow: Start, then the policy agent, then Replywriter.
3. Click the policy agent's step, open **Node settings**, and under **Save agent output message as** create a new variable named **Local.PolicyAnswer**.
4. Click Replywriter's step, open **Node settings**, and set **Input message** to **Local.PolicyAnswer**. **This step is essential:** without it Replywriter receives an empty message and replies with nonsense (we saw "0 ->").
5. Click **Save**, then **Preview**. The **YAML** tab shows the same workflow as text: the policy step has *messages: Local.PolicyAnswer* under output, and Replywriter has *messages: =Local.PolicyAnswer* under input.

Demo questions, in the workflow preview

Question	What it shows
I've had 3 refunds this month, can I claim another?	The policy agent finds the three-claims rule. Replywriter writes "Dear customer, ... a fourth claim within that window is automatically declined ... FreshCart Support".
Do you refund grocery orders in Singapore?	The policy agent quotes the Scope section (orders in India). Replywriter says Singapore grocery orders are not covered and apologises, with no made-up policy.

The teaching point. The second agent is where made-up facts slip back in. With an earlier, looser instruction, the policy agent wrote that the policy applies to India, and Replywriter turned that into "refunds for Singapore are not authorized": a decision the policy never makes. The fix was in the instructions of both agents. Click **Traces** under each reply to show the class what each agent received and returned.

Note for the course. The portal says Foundry is retiring workflows on 1 December 2026, in favour of Microsoft Agent Framework. This demo is fine until then, but do not build course labs on Foundry workflows.

If something goes wrong

What you see	What to do
MCP tool: Authentication failed ... 401 (Unauthorized)	Foundry is sending no key or the wrong one. Use the mcp_extension key from System keys, in the address after ?code= or in the header x-functions-key.
MCP address answers 403	You used the default key from Part 3. The MCP address needs mcp_extension.
Functions list shows only get_metric	The MCP add-on did not load. Check host.json uses the Experimental bundle shown in Part 4, then deploy again.
OpenAPI tool: 401	Check the header name is exactly x-functions-key and the value is the default host key.
The agent uses the other tool than you asked for	The two functions share a name. Check openapi.json has operationId get_metric_openapi, paste it into the tool again, and save.
A change seems to do nothing	Click Save in the agent. The playground runs the saved version.

The agent invents a number	The instruction line for numbers is missing. Add line 3 from Part 3 and save.

After class. Delete the Function App in the Azure portal, or at least its keys, so the addresses stop working. Never put a key in a slide, a chat or a shared file.